

Test Plan

Proyecto 1 – Bus con Arbitraje Round-Robin

Wilbert Roman - 2019042296

David Leiton - 2021103803

15 de septiembre de 2026

1. Objetivo

Verificar funcionalmente el DUT `prll_bs_gnrtr_n_rbtr_fifo`: bus full-mesh entre `drvrs` dispositivos con arbitraje round-robin, demostrando entrega correcta de paquetes, rotación del árbitro, y manejo de direcciones inválidas/broadcast bajo múltiples configuraciones de `drvrs`.

2. Parámetros del DUT

Parámetro	Significado	Valores
<code>bits</code>	Ancho de payload	8, 16, 32
<code>drvrs</code>	Número de dispositivos	2, 4, 8
<code>broadcast</code>	Dirección de broadcast	$\{8\{1'b1\}\}$
<code>depth</code>	Profundidad de FIFO	2, 4, 8

3. Arquitectura del ambiente

El Agent/Generator **no envía la transacción directamente al Checker**: la entrega al Driver, que la aplica sobre la interfaz (señales `push/D_push`) hacia el DUT. El Checker recibe lo que realmente ocurrió a través del **Monitor**, que observa las señales del DUT (`D_pop/pndng`), no a través del Driver.

4. Interfaces de comunicación entre módulos

Mailbox	Emisor	Receptor	Paquete
<code>tst_agnt_mbx</code>	Test	Agent/Generator	<code>instrucciones_agente</code>
<code>tst_sb_mbx</code>	Test	Score_board	<code>solicitud_sb</code>
<code>agnt_drv_mbx</code>	Agent/Generator	Driver (Manejador, hijo)	<code>trans_bus</code>
–	Driver ↔ Interfaz ↔ DUT	(señales directas: <code>push</code> , <code>D_push</code> , <code>pop</code> , <code>D_pop</code> , <code>pndng</code> ; sin mailbox)	–
<code>mon_chkr_mbx</code>	Monitor (observa el DUT)	Checker	<code>trans_bus</code> observada
<code>chkr_sb_mbx</code>	Checker	Score_board	<code>trans_sb</code>

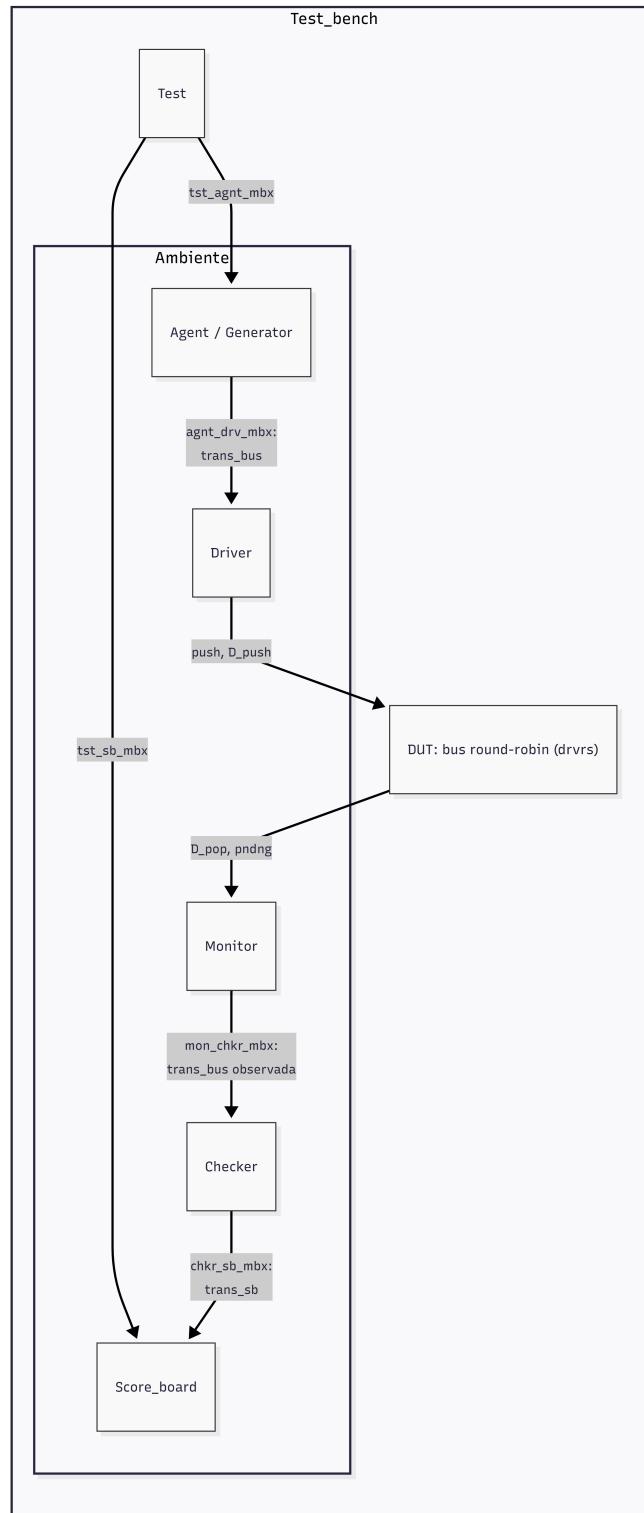


Figura 1: Test_bench: Test, Ambiente (Agent/Generator, Driver, Monitor, Checker, Score_board) y DUT, con los mailboxes de comunicación entre bloques.

5. Formato de los paquetes

trans_bus (Agent/Generator → Driver → DUT → Monitor → Checker)

Campo	Tipo	Descripción
origen	int	Dispositivo origen (0..drvrs-1)
destino	int	Dispositivo destino o broadcast
payload	bit[bits-1:0]	Dato a transmitir
tipo	enum {valida, broadcast, invalida}	Escenario
pckg_sz	int	16, 32 o 64
retardo	int	Ciclos antes de ejecutar

trans_sb (Checker → Score_board)

Campo	Tipo	Descripción
dato_enviado	bit[bits-1:0]	Payload observado
t_envio / t_recibido	int	\$time de envío/recepción
latencia	int	t_recibido - t_envio
resultado	enum {completado, perdido, overflow, underflow}	Resultado del matching

6. Tabla de casos de prueba

Nombre	Qué se prueba	Qué hace	Qué se aleatoriza y rangos
test_general_regresion	F0: caso general – cubre F1–F9 y F12 a la vez	Tráfico constrained-random continuo entre todos los dispositivos, con múltiples semillas (+SEED).	Origen/destino: 0 a drvrs-1; payload: rango completo de bits; pckg_sz: uniforme; tiempo entre transacciones: 0 a N ciclos.
test_push_basico	F1: push exitoso	Escribe N transacciones en FIFOs con espacio y verifica orden FIFO.	Payload: 0 a $2^{bits} - 1$; dispositivo: 0 a drvrs-1.
test_pop_basico	F2: pop exitoso	Escritura seguida de lectura, verifica dato leído = esperado.	Transacciones por dispositivo: 1 a 20; retardo push-pop: 0 a 10 ciclos.
test_fifo_full	F3: FIFO llena (full)	Llena hasta depth; push adicional no corrompe estado.	depth $\in \{2, 4, 8\}$.
test_fifo_empty	F4: FIFO vacía (pndng)	Verifica pndng=0 y pop sobre vacía no genera underflow.	Dirigido, sin aleatorización.

Nombre	Qué se prueba	Qué hace	Qué se aleatoriza y rangos
<code>test_arb_full_load</code>	F5–F8: arbitraje round-robin (1, 2 y todos los requesters)	Verifica orden de concesión mediante cross coverage ganador actual $\times$ anterior, para cada <code>drvrs</code> .	<code>drvrs</code> $\in \{2, 4, 8\}$; rondas observadas: 10 a 50.
<code>test_valid_transfer</code>	F9: entrega origen $\rightarrow$ destino válido	Tráfico válido entre pares; verifica integridad en scoreboard por par.	Origen/destino: 0 a <code>drvrs</code> –1, destino $\neq$ origen; payload según <code>bits</code> .
<code>test_invalid_addr</code>	F10: dirección inexistente (caso de esquina)	Fuerza destino fuera de rango; verifica pérdida por timeout del scoreboard.	Dirigido: destino fijo fuera de rango, origen barrido 0 a <code>drvrs</code> –1.
<code>test_broadcast</code>	F11: broadcast (caso de esquina)	Envía a <code>broadcast</code> ; verifica recepción en todos menos el origen.	Dirigido: origen barrido 0 a <code>drvrs</code> –1.
<code>test_pckg_sizes</code>	F12: integridad según tamaño de paquete	Repite tráfico válido variando <code>bits</code> .	<code>bits</code> $\in \{8, 16, 32\}$.
<code>test_reset_mtdtransfer</code>	F13: reset en transacción pendiente (caso de esquina)	Reset con datos en tránsito; verifica retorno a estado limpio.	Dirigido: bajo/medio/alto llenado de FIFO.
<code>test_fifo_fill_drain</code>	F14: ciclo llenado completo $\rightarrow$ vaciado completo (caso de esquina)	Llena la FIFO hasta <code>depth</code> sin hacer pop, luego la vacía por completo sin hacer push; verifica que <code>full</code> y <code>pndng</code> cambian exactamente en el límite correcto (<code>count=depth</code> y <code>count=0</code>).	Dirigido: <code>depth</code> $\in \{2, 4, 8\}$; dato de cada push: aleatorio.
<code>test_push_pop_simult</code>	F15: push y pop simultáneos (caso de esquina)	Genera <code>push=1</code> y <code>pop=1</code> en el mismo ciclo con la FIFO vacía, a nivel bajo, medio, alto y llena; verifica que <code>count</code> no se corrompe (rama 2'b11 del RTL).	Dirigido: nivel de llenado $\in \{\text{vacía, bajo, medio, alto, llena}\}$; payload: aleatorio.
<code>test_arb_counter_wrap</code>	F16: wrap-around del contador de arbitraje (caso de esquina)	Fuerza que <code>Counter_arb</code> llegue a <code>drvrs</code> –1 y vuelva a 0 con una solicitud pendiente; verifica que el dispositivo 0 recibe concesión justa en el wrap, sin salto ni repetición.	Dirigido: número de rondas forzadas = <code>drvrs</code> , para cada <code>drvrs</code> $\in \{2, 4, 8\}$.